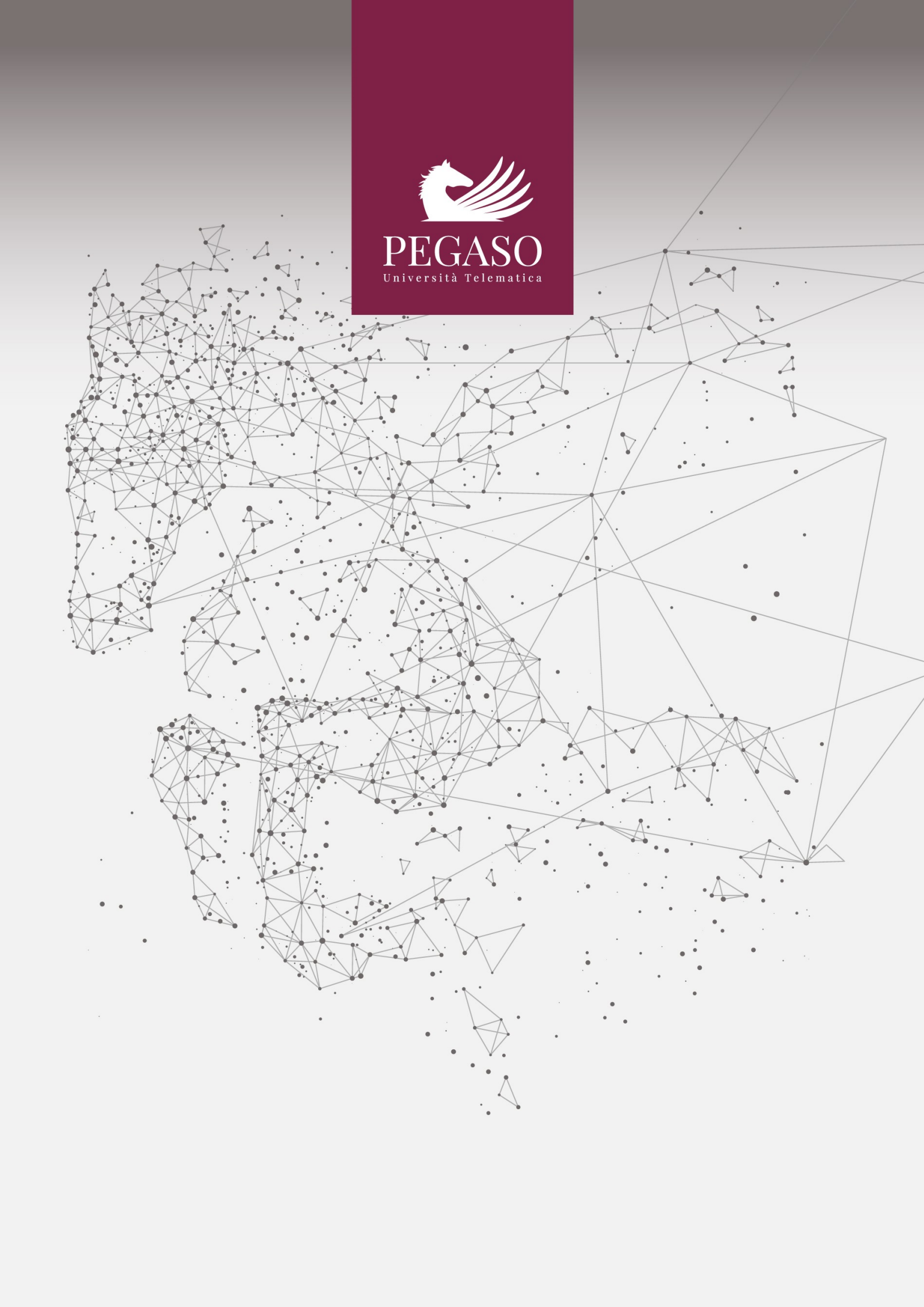

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Indice

**1. OPERATORI LOGICI AND, OR E DI SHIFT ..... 3**

**2. ASSEMBLY MIPS: LE ISTRUZIONI AND, OR, SLL, SRL ..... 6**

**3. FORMATO TIPO R E INDIRIZZAMENTO PER LE ISTRUZIONI LOGICHE E DI SHIFT ..... 10**

**BIBLIOGRAFIA ..... 17**

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 1. Operatori logici AND, OR e di shift

Gli operatori Logici sono funzioni che hanno come operandi valori binari ai quali vengono associati i significati di valori di verità: "vero", rappresentato dal simbolo 1, e "falso", rappresentato dal simbolo 0. Il risultato è anch'esso un valore di verità.

Come tutte le funzioni un operatore Logico è definito dalla tabella che stabilisce le corrispondenze tra i valori degli operandi e il risultato, che in questo caso prendono il nome di *tavola di verità*.
Qui di seguito vengono riportate le tavole di verità degli operatori Logici AND e OR:

| x | y | AND(x, y) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 1 | 0 | 0 |
| 1 | 1 | 1 |

| x | y | OR(x, y) |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

le Tavole mostrano i seguenti significati degli operatori Logici:

AND valore "vero" solo quando entrambi gli operandi hanno valore "vero"

OR valore "falso" solo quando entrambi gli operandi hanno valore "falso"

Gli operatori Logici possono essere composti per costruire espressioni logiche che vengono valutate in funzione dei valori delle variabili utilizzando le tavole di verità degli operatori.

Nella programmazione gli operatori Logici sono utilizzati per costruire espressioni logiche rappresentanti condizioni che determinano scelte diverse a seconda del valore di verità risultante dalla loro valutazione.

Un'ulteriore utilizzazione degli operatori Logici è per l'elaborazione dei singoli bit di una sequenza. In questo caso si parla di *operatori Logici bit a bit.*

Usando l'AND bit a bit si può forzare a 0 il valore di specifici bit di una sequenza lasciando invariati i rimanenti come mostrato dai seguenti esempi:

| | | |
|---|---|---|
| *primo operando* | 10110110 | 10110110 |
| *secondo operando* | 1111**0000** | **0000**1111 |
| **AND** bit a bit | 1011**0000** | **0000**0110 |

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Gli esempi mostrano che ponendo a 0 un bit del secondo operando il risultato dell'AND sarà 0 in quella posizione, invece ponendo il bit a 1 il risultato darà il valore del corrispondente bit del primo operando in quella posizione. In questo caso si dice che il secondo operando è una *maschera* in quanto con il valore 0 nasconde il valore del bit corrispondente del primo operando ponendolo a zero, mentre con il valore 1 lo lascia immutato.

Invece, per forzare ad 1 il valore di specifici bit di una sequenza lasciando immutati gli altri si può utilizzare l'OR bit a bit come mostrato dai seguenti esempi:

| | | |
|---|---|---|
| *primo operando* | 10100100 | 10100100 |
| *secondo operando* | **1111**0000 | 0000**1111** |
| **OR** bit a bit | **1111**0100 | 1010**1111** |

Gli esempi mostrano che in maniera duale (cioè scambiando "vero" con "falso" e viceversa) ponendo a 1 un bit del secondo operando il risultato dell'OR sarà 1 in quella posizione, invece ponendo il bit a 0 il risultato darà il valore del corrispondente bit del primo operando in quella posizione.

Un ulteriore operatore per l'elaborazione dei bit di una sequenza di lunghezza fissata è lo SHIFT (scorrimento) che sposta di un certo numero di posizioni tutti i bit della sequenza a sinistra o a destra mantenendo il loro ordine. I bit nelle posizioni lasciate vuote sono posti uguali a 0, quelli che per effetto dello scorrimento escono dalla sequenza sono perduti.

Le operazioni di *shift*, insieme alle operazioni Logiche bit a bit, sono utilizzate per elaborazioni combinatorie delle sequenze binarie.

Inoltre l'operazione di **SHIFT *a sinistra*** di sequenze binarie può essere utilizzata per moltiplicare il valore del numero positivo rappresentato in Notazione posizionale pesata per potenze di 2 se i bit che si perdono nello scorrimento sono uguali a 0. La moltiplicazione si ottiene effettuando un numero di spostamenti uguale all'esponente come mostrato nel seguente esempio:

00000101 ⟷ 5

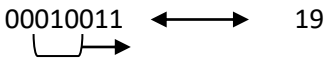
applicando lo SHIFT a sinistra di 3 posizioni e aggiungendo tre 0 a destra nelle posizioni lasciate vuote si ha:

00101000 ⟷ $5 \times 2^3 = 5 \times 8 = 40$.

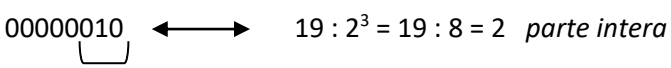
PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Simmetricamente, l'operazione di **SHIFT *a destra*** di sequenze binarie può essere utilizzata per dividere un intero positivo per potenze di 2 ottenendo come risultato la parte intera del quoziente come mostrato nel seguente esempio:




applicando lo SHIFT a destra di 3 posizioni e aggiungendo tre 0 a sinistra nelle posizioni lasciate vuote si ha:




Le operazioni di moltiplicazione e divisione eseguite con le istruzioni corrispondenti agli operatori di *shift* sono più veloci di quelle eseguite con le istruzioni corrispondenti agli operatori aritmetici.

Queste proprietà sono applicate anche nell'uso comune della notazione decimale per effettuare le moltiplicazioni per potenze di 10 aggiungendo un numero di zeri a destra uguale all'esponente, o spostando la virgola a sinistra per la divisione.

Nel caso della moltiplicazione, però, nell'uso comune la lunghezza della sequenza viene aumentata con l'aggiunta degli 0 a destra senza perdere le cifre del numero che si spostano a sinistra.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 2. Assembly MIPS: le istruzioni AND, OR, SLL, SRL

Per evitare ambiguità, le istruzioni sono scritte seguendo le regole precise di un *Linguaggio Formale*. Tali regole costituiscono la *Sintassi* del Linguaggio, ed in base ad esse la macchina determina univocamente le azioni che dovranno esse svolte dall'Hardware.

STRUTTURA DI BASE DI UNA ISTRUZIONE ASSEMBLY MIPS

- Uso dei simboli dell'alfabeto inglese e della notazione decimale.
- Ogni istruzione è scritta su una sola riga.
- Ogni istruzione contiene:
    - una sequenza alfanumerica iniziale, detta **Codice Operativo**, che dal punto di vista semantico rappresenta il tipo di azione da eseguire e come reperire gli operandi,
    - altri simboli che consentono di stabilire dove si trovano gli operandi e dove memorizzare il risultato.

È possibile associare un **commento** ad un'istruzione ponendolo sulla stessa riga dell'istruzione e facendolo iniziare con il simbolo "#". Se il commento occupa più righe, ogni riga deve iniziare con il simbolo "#".

Le **istruzioni Logiche bit a bit dell'Assembly** MIPS con Codici Operativi *and* ed *or* hanno due operandi rappresentati da sequenze di 32 bit contenute in registri del processore. Eseguono operazioni logiche AND ed OR sui bit che occupano posizioni corrispondenti. I bit ottenuti costituiscono la sequenza di 32 bit del risultato che viene scritta in un registro del processore.

Per individuare i registri che contengono gli operanti ed il risultato, all'interno dell'istruzione viene fornito il rispettivo indirizzo in precise posizioni.

I valori degli operandi devono essere già presenti nei registri prima di eseguire l'istruzione. Il registro in cui viene memorizzato il risultato è detto *registro destinazione*.

Poiché tali istruzioni coinvolgono l'uso dei registri, riportiamo la tabella con gli indirizzi dei registri del processore utilizzati per i dati, indicando gli indirizzi numerici in Notazione decimale e la corrispondente notazione simbolica utilizzata nell'Assembly MIPS:

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

| *simbolo* | *indirizzo* | *simbolo* | *indirizzo* | *simbolo* | *indirizzo* |
|---|---|---|---|---|---|
| $t0 | 8 | $s0 | 16 | $t8 | 24 |
| $t1 | 9 | $s1 | 17 | $t9 | 25 |
| $t2 | 10 | $s2 | 18 | | |
| $t3 | 11 | $s3 | 19 | | |
| $t4 | 12 | $s4 | 20 | | |
| $t5 | 13 | $s5 | 21 | | |
| $t6 | 14 | $s6 | 22 | | |
| $t7 | 15 | $s7 | 23 | | |

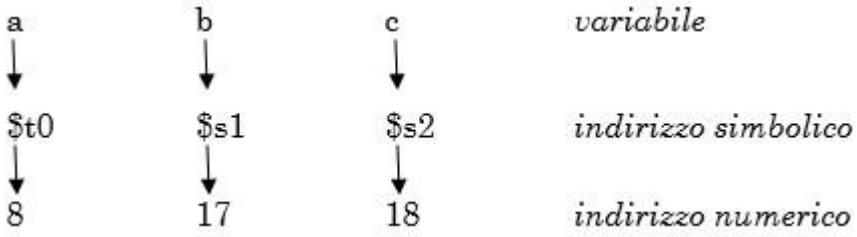
Gli operatori AND e OR bit a bit sono disponibili anche in Linguaggi ad Alto Livello. Nei Linguaggi C, C++ e Java sono denotati, rispettivamente, con i simboli "&" e "|".

Il seguente esempio di assegnamento in Linguaggio C è relativo **all'AND bit a bit**:

```
a = b & c  ;
```

La sintassi delle istruzioni Logiche bit a bit *and* e *or* è analoga a quella delle istruzioni Aritmetiche *add* e *sub*.

Quindi, assegnando alle variabili "a", "b", "c" i registri con i seguenti INDIRIZZI:




l'istruzione in C viene tradotta nell'**istruzione Assembly MIPS**:

```
and  $t0, $s1, $s2
```

Vincoli sintattici:

- Codice Operativo **and**,
- l'**indirizzo del registro DESTINAZIONE** che conterrà il risultato è il primo, ed è separato da una virgola dai successivi indirizzi,

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

- gli **indirizzi dei due REGISTRI DEGLI OPERANDI** seguono l'indirizzo destinazione e sono separati da una virgola.

L'esecuzione dell'istruzione ha effetto sul contenuto del registro destinazione, ma lascia immutato il contenuto dei registri degli operandi.

Analogamente, per l'operatore **OR bit a bit** dato dall'istruzione in Linguaggio C:

```
a = b | c  ;
```

usando la stessa assegnazione di registri **in Assembly MIPS** si ha:

```
or  $t0,  $s1,  $s2
```

con gli stessi vincoli sintattici.

Le **istruzioni di SHIFT dell'Assembly** MIPS con Codici Operativi *sll* e *srl* hanno un solo operando rappresentato da una sequenza di 32 bit contenuta in un registro del processore. Eseguono operazioni di *shift* spostando i bit dell'operando a sinistra (*sll* **s**hift **l**eft **l**ogical) o a destra (*srl* **s**hift **r**ight **l**ogical). I bit nelle posizioni lasciate vuote sono posti uguali a 0, quelli che per effetto dello scorrimento escono dalla sequenza sono perduti.

Il numero di spostamenti è indicato nell'istruzione. La sequenza di 32 bit ottenuta come risultato viene scritta in un registro del processore.

Gli operatori di SHIFT *a sinistra* e *a destra* sono disponibili anche in Linguaggi ad Alto Livello. Nei Linguaggi C, C++ e Java sono denotati, rispettivamente, con i simboli "<<" e ">>".

Il seguente esempio di assegnamento in Linguaggio C ha come risultato lo **SHIFT *a sinistra*** di 3 posizioni dei bit contenuti nella variabile b:

```
a = b << 3  ;
```

Assegnando alla variabile "b" dell'operando ed alla variabile "a" del risultato i registri con i seguenti INDIRIZZI:

| | | |
|---|---|---|
| a | b | *variabile* |
| ↓ | ↓ | |
| $t1 | $s0 | *indirizzo simbolico* |
| ↓ | ↓ | |
| 9 | 16 | *indirizzo numerico* |

l'istruzione in C viene tradotta nell'**istruzione Assembly MIPS**:

```
sll  $t1,  $s0,  3
```

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

Vincoli sintattici:

- Codice Operativo **sll**,
- l'**indirizzo del registro DESTINAZIONE** che conterrà il risultato è il primo, ed è separato da una virgola dal successivo indirizzo,
- l'**indirizzo dell'OPERANDO CONTENUTO NEL REGISTRO** segue l'indirizzo del registro destinazione ed è separato da una virgola dalla successiva costante,
- il **valore del NUMERO DI POSIZIONI dello *shift*** è l'ultima informazione rappresentata da un numero positivo che può essere scritto con la **usuale notazione decimale**.

L'esecuzione dell'istruzione ha effetto sul contenuto del registro destinazione, ma lascia immutato il contenuto del registro dell'operando.

Analogamente, per l'operatore **SHIFT *a destra*** dato dall'istruzione in Linguaggio C:

```
a = b >> 5  ;
```

usando la stessa assegnazione di registri **in Assembly MIPS** si ha:

```
srl  $t0,  $s1,  5
```

con gli stessi vincoli sintattici.

Le regole sintattiche guidano il processo di traduzione di un'istruzione del Linguaggio Assembly nella sequenza binaria che rappresenta la corrisponde istruzione in Linguaggio Macchina.

Nel successivo paragrafo vengono descritte le regole sintattiche delle istruzioni del Linguaggio Macchina MIPS corrispondenti alle istruzioni Assembly MIPS relative alle operazioni Logiche bit a bit e di shift.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# 3. Formato Tipo R e Indirizzamento per le istruzioni logiche e di shift

La sequenza binaria che rappresenta una istruzione in **LINGUAGGIO MACCHINA** permette di ricavare tutte le informazioni che definiscono sia l'azione associata all'istruzione da eseguire, sia le modalità per il reperimento dei dati.

Queste informazioni sono codificate mediante due fondamentali nozioni che sono alla base della definizione di tutti i Linguaggi Macchina e che caratterizzano ogni istruzione:

- il Formato dell'istruzione
- la Modalità di Indirizzamento dell'istruzione.

La SEQUENZA BINARIA che rappresenta l'istruzione in Linguaggio Macchina è suddivisa in sottosequenze, dette CAMPI, di lunghezza e posizione fissata i cui contenuti forniscono specifiche informazioni determinate univocamente dalle regole sintattiche dell'istruzione. Questa strutturazione, detta **FORMATO**, è solo una organizzazione concettuale e non determina nessuna suddivisione a livello fisico della sequenza.

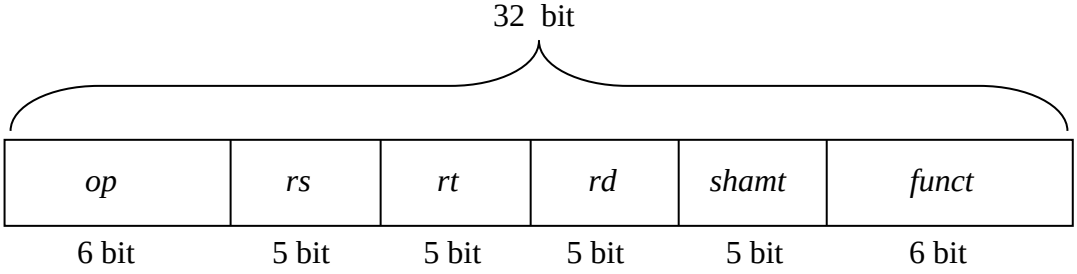
Il **Formato di Tipo R** deve la sua denominazione al fatto che gli operandi sono contenuti in registri del processore e gli indirizzi che li individuano sono forniti nei campi dell'istruzione. La lettera R rappresenta, quindi, l'iniziale della parola *Registro*.

Gli indirizzi sono numeri interi positivi rappresentati da 5 bit in Notazione posizionale pesata.

Ogni istruzione del Linguaggio Macchina MIPS è rappresentata da una sequenza di 32 bit.

Il Formato di Tipo R suddivide i 32 bit della sequenza che rappresenta l'istruzione in 6 sottosequenze in base alla seguente partizione (concettuale, non fisica) in **CAMPI** di lunghezze e posizioni fissate. Le sottosequenze contenute nei campi sono denotate da nomi simbolici utilizzati per descrivere il loro utilizzo nelle istruzioni che usano questo formato:

**FORMATO di Tipo R**




*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

*op*: 6 bit, campo *Codice Operativo* presente in tutte le istruzioni. Il valore contenuto indica l'operazione da eseguire ed il formato dell'istruzione. Nelle istruzioni Aritmetico-Logiche contiene il valore 0 che indica che il Formato è di Tipo R e che l'operazione relativa all'istruzione è specificata nel campo *funct*;

*rs*: 5 bit, indirizzo di registro. Nelle istruzioni Aritmetico-Logiche il corrispondente registro contiene il primo operando;

*rt*: 5 bit, indirizzo di registro. Nelle istruzioni Aritmetico-Logiche il corrispondente registro contiene il secondo operando;

*rd*: 5 bit, indirizzo di registro. Nelle istruzioni Aritmetico-Logiche il corrispondente registro contiene il risultato, detto *registro destinazione*;

*shamt*: 5 bit, nelle istruzioni di *shift* contiene il numero di posizioni di scorrimento della sequenza (*shift amount*), nelle istruzioni Aritmetico-Logiche contiene il valore 0;

*funct*: 6 bit, nelle istruzioni Aritmetico-Logiche specifica l'operazione (*function*) relativa all'istruzione.

Le istruzioni del **LINGUAGGIO MACCHINA MIPS** che corrispondono alle istruzioni LOGICHE bit a bit dell'Assembly MIPS *and* e *or* utilizzano il Formato di Tipo R.

Le regole sintattiche che guidano la costruzione della sequenza di 32 bit che rappresenta l'istruzione in Linguaggio Macchina sono le stesse per entrambe le istruzioni. Vengono quindi descritte facendo riferimento all'istruzione *and* e si applicano analogamente ad *or* con la sola differenza del valore contenuto nel campo *funct*.

Nell'istruzione ***and*** **AND bit a bit dell'Assembly MIPS** i valori degli operandi e del risultato sono contenuti in registri del processore.

Considerando ad esempio l'istruzione:

```
and  $t0,  $s1,  $s2
```

gli indirizzi dei registri degli operandi sono $s1, $s2, l'indirizzo del registro destinazione che contiene il risultato è $t0.

Il campo *Codice Operativo* (*op*) della corrispondente istruzione del Linguaggio Macchina contiene il valore 0 rappresentato da una sequenza di 6 bit tutti uguali a 0. In base a questo valore la macchina stabilisce che il Formato è di Tipo R e che si tratta di una istruzione che esegue una operazione Aritmetico-Logica.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

La specifica operazione logica eseguita dall'istruzione è indicata dal valore del campo *funct*. La seguente tabella riporta i valori corrispondenti alle due istruzioni Logiche bit a bit:

| istruzione | 6 bit del campo *funct* | |
|---|---|---|
| | decimale | binario |
| *and* | 36 | 100100 |
| *or* | 37 | 100101 |

In base al Formato sono automaticamente determinati i campi che contengono le informazioni necessarie all'esecuzione.

Per completare la rappresentazione delle informazioni necessarie all'esecuzione dell'istruzione occorre fornire i tre indirizzi dei registri che contengono gli operandi ed il risultato. Ogni indirizzo è un intero positivo rappresentato da sequenze di 5 bit. Questi indirizzi vengono posti nei campi *rs, rt* ed *rd* del Formato di Tipo R.

Per l'esempio di istruzione *and* considerato, in base alla tabella delle corrispondenze tra notazione simbolica e valore dell'indirizzo si ha che gli indirizzi degli operandi $s1, $s2 e del registro destinazione $t0 che contiene il risultato sono dati dalle seguenti sequenze di 5 bit:

$s1 ⟷ 17 ⟷ 10001
$s2 ⟷ 18 ⟷ 10010
$t0 ⟷ 8 ⟷ 01000

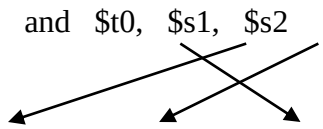
Il campo *shamt* non è utilizzato e viene posto uguale a 0 poiché non esiste la condizione "vuoto" per il contenuto di un registro.

Tutte queste informazioni occupano specifici campi del Formato di Tipo I componendo la sequenza di 32 bit che rappresenta l'istruzione in Linguaggio Macchina.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

L'ordine degli indirizzi dei registri viene invertito come mostrato dal seguente schema:




| *op* | *rs* | *rt* | *rd* | *shamt* | *funct* |
|---|---|---|---|---|---|
| 0 | 17 | 18 | 8 | 0 | 36 |

la cui rappresentazione binaria è:

| 000000 | 10001 | 10010 | 01000 | 00000 | 100100 |
|---|---|---|---|---|---|

e quindi la sequenza dei 32 bit in Linguaggio Macchina MIPS è:

**00000010001100100100000000100100**

Anche le istruzioni del **LINGUAGGIO MACCHINA** che corrispondono alle istruzioni **SHIFT *a sinistra* e SHIFT *a destra*** dell'Assembly MIPS *sll* ed *srl* utilizzano il Formato di Tipo R.

Le regole sintattiche sono le stesse per entrambe le istruzioni. Vengono quindi descritte facendo riferimento all'istruzione *sll* e si applicano analogamente ad *srl* con la sola differenza del valore contenuto nel campo *funct*.

Nell'istruzione ***sll* SHIFT bit a bit a sinistra dell'Assembly MIPS** i valori dell'operando e del risultato sono contenuti in registri del processore.

Considerando ad esempio l'istruzione:

```
sll  $t0, $s2, 7
```

l'indirizzo del registro dell'operando è $s2, l'indirizzo del registro destinazione che contiene il risultato è $t0.

Il campo *Codice Operativo* (*op*) della corrispondente istruzione del Linguaggio Macchina contiene il valore 0 rappresentato da una sequenza di 6 bit tutti uguali a 0. In base a questo valore la macchina stabilisce che il Formato è di Tipo R e che si tratta di una istruzione che esegue una operazione Aritmetico-Logica.

La specifica operazione logica eseguita dall'istruzione è indicata dal valore del campo *funct*. La seguente tabella riporta i valori corrispondenti alle due istruzioni Logiche bit a bit:

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

| istruzione | 6 bit del campo *funct* | |
|---|---|---|
| | decimale | binario |
| *sll* | 0 | 000000 |
| *srl* | 2 | 000010 |

In base al Formato sono automaticamente determinati i campi che contengono le informazioni necessarie all'esecuzione.

Per completare la rappresentazione delle informazioni necessarie all'esecuzione dell'istruzione occorre fornire i due indirizzi dei registri che contengono l'operando ed il risultato. Ogni indirizzo è un intero positivo rappresentato da sequenze di 5 bit. Questi indirizzi vengono posti nei campi *rt* ed *rd* del Formato di Tipo R.

Per l'esempio di istruzione *sll* considerato, in base alla tabella delle corrispondenze tra notazione simbolica e valore dell'indirizzo si ha che l'indirizzo dell'operando $s2 e del registro destinazione $t0 che contiene il risultato sono dati dalle seguenti sequenze di 5 bit:

$s2 ⟷ 18 ⟷ 10010

$t0 ⟷ 8 ⟷ 01000.

Il campo *rs* non è utilizzato e viene posto uguale a 0 poiché non esiste la condizione "vuoto" per il contenuto di un registro.

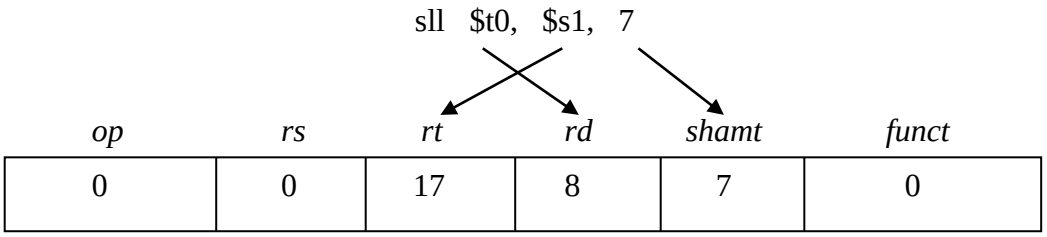
Nel campo *shamt* (*shift amount*) viene posto l'intero positivo che rappresenta il numero di posizioni da scalare indicato nell'istruzione Assembly.

I cinque bit del campo *shamt* possono rappresentare un numero intero positivo minore o uguale di $31 = 2^5 - 1$. Questi valori sono sufficienti per rappresentare *shift* significativi perché uno *shift* di un numero di posizioni maggiore o uguale a 32 produce una sequenza non significativa di tutti 0. Per l'esempio di istruzione *sll* considerato l'ammontare dello *shift* è 7.

Tutte queste informazioni occupano specifici campi del Formato di Tipo I componendo la sequenza di 32 bit che rappresenta l'istruzione in Linguaggio Macchina. L'ordine degli indirizzi dei registri viene invertito come mostrato dal seguente schema:

PEGASO Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*




la cui rappresentazione binaria è:

| | | | | | |
|---|---|---|---|---|---|
| 000000 | 00000 | 10001 | 01000 | 00111 | 000000 |

e quindi la sequenza dei 32 bit in Linguaggio Macchina MIPS è:

**00000000000010001010000011100000 0**

Il Formato di Tipo R viene usato da tutte le istruzioni con una **unica Modalità di Indirizzamento**:

**INDIRIZZAMENTO tramite registro**

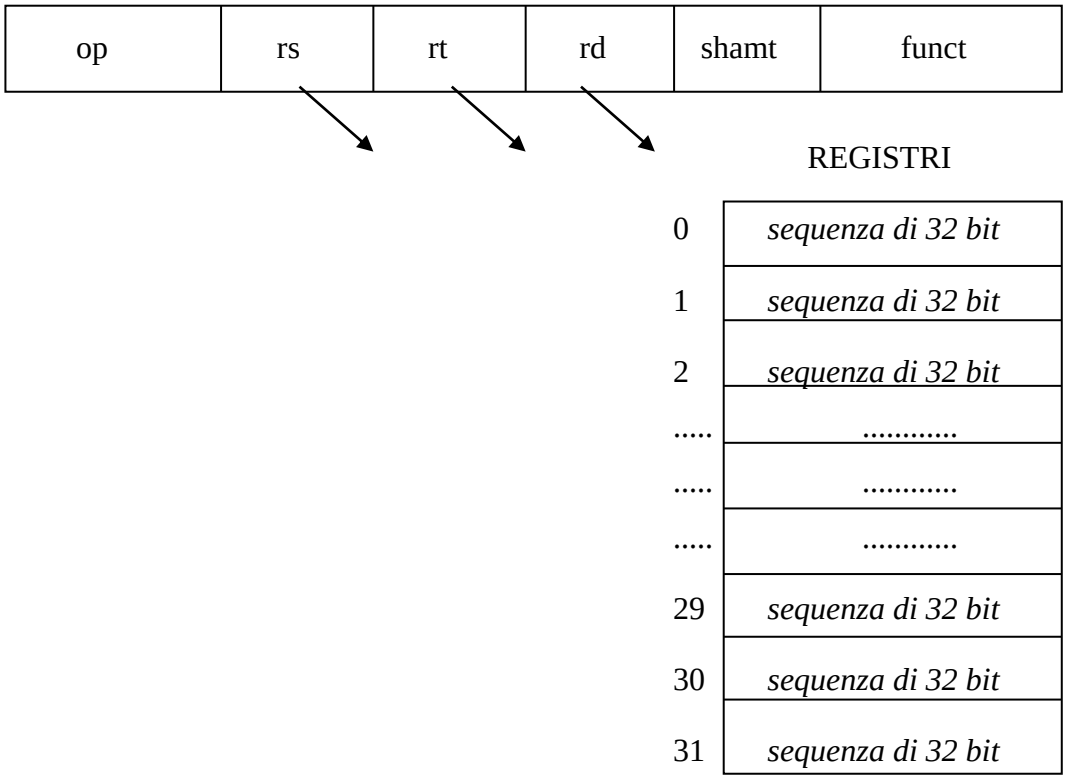
I campi di 5 bit *rs*, *rt*, *rd* del Formato di Tipo R contengono indirizzi di registri del processore in cui sono memorizzati valori di dati coinvolti nella esecuzione dell'istruzione.

PEGASO
Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

La seguente figura mostra una rappresentazione schematica del significato di tale Modalità di Indirizzamento:

**INDIRIZZAMENTO tramite registro**




La seguente tabella riassume le regole sintattiche relative alle istruzioni presentate:

**Linguaggio Macchina MIPS**

| | | 6 bit | 5 bit | 5 bit | 5 bit | 5 bit | 6 bit | |
|---|---|---|---|---|---|---|---|---|
| *istruzione* | *Tipo* | *op* | *rs* | *rt* | *rd* | *shamt* | *funct* | esempio |
| and | R | 0 | $s1 | $s2 | $t0 | 0 | 36 | and $t0, $s1, $s2 |
| or | R | 0 | $s1 | $s2 | $t0 | 0 | 37 | or $t0, $s1, $s2 |
| sll | R | 0 | 0 | $s2 | $t0 | 7 | 0 | sll $t0, $s2, 7 |
| srl | R | 0 | 0 | $s2 | $t0 | 7 | 2 | sll $t0, $s2, 7 |

PEGASO Università Telematica

*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*

# Bibliografia


- D. A. Patterson, J. L. Hennessy, "Struttura e progetto dei calcolatori", Zanichelli.


*Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).*